

Gaussian Parity Certificates for Exact Butterfly Arithmetic

A Lean-checked interface for integer-multiplication networks

Alejandro Zarzuelo Urdiales
AI-assisted research and formalization

8 October 2026

Abstract

The communal effort to improve integer multiplication below $n \log n$ uses exact Gaussian dyadic butterfly networks. Its arithmetic certificates benefit from distinguishing a completed transform’s return lattice from the grids needed by unfinished intermediate values. We give a small, executable, Lean-checked interface for this distinction. For the synthetic phase kernel C , division is integer-exact precisely when the two Gaussian integer numerators at a common denominator exponent have the same one-bit residue modulo $1 + i$. A total fallback retains the missing denominator instead of discarding it. A completed D -axis C tensor has sharp additional binary denominator exponent $\lceil D/2 \rceil$, with a parity-constrained numerator lattice for odd D . The final normalized Hadamard tensor still has sharp exponent D . Counterexamples certify why an initial parity promise cannot authorize every later division or the final normalization. The contribution is a reusable precision-audit component, not a new multiplication exponent or a formal verification of the full tape-machine construction.

1 The integration problem

OpenAI’s manuscript [1] and the conditional extensions developed by Douglas Colkitt, Swapnil Jain, and other contributors [2, 3, 4] implement simultaneous butterfly layers through recursive arithmetic and address networks. Two obligations should be separated: a completed child must return the intended linear operator, and every arithmetic intermediate must be represented exactly. The Gaussian kernel in the manuscript is

$$C = \frac{1}{2} \begin{pmatrix} 1+i & 1-i \\ 1-i & 1+i \end{pmatrix} = \frac{1}{\varpi} \begin{pmatrix} i & 1 \\ 1 & i \end{pmatrix}, \quad \varpi = 1 + i. \quad (1)$$

The normalized Hadamard kernel is

$$H_0 = \frac{1}{2} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} = bSCS, \quad b = \varpi^{-1}, \quad S = \text{diag}(1, i). \quad (2)$$

The pinned upstream source gives these formulas in its phase-kernel identities and its individual pair evaluation.

Provenance and development. An earlier Archivara exploration of parity-aware integer multiplication initiated this research. Alejandro Zarzuelo Urdiales developed it personally with multiple AI tools, most recently GPT-6.1 as recorded by the author. OpenAI Codex assisted the present paper, Lean proofs, and exact controls. These results have independent proofs and do not rely on the earlier draft’s broader claims.

The original motivation was to exploit powers of two before integer multiplication. For this initiative, a more relevant refinement is to exploit the ramification $\varpi^2 = 2i$ inside its actual complex arithmetic. The one-bit residue below decides a legal exact division, rather than asserting that discarded carry information can later be recovered for free.

Gaussian denominator exponents and parity-based reduction are established tools in exact circuit synthesis [7, Section 5.4, Eq. (11), Lemmas 5.23–5.24]. We claim no new Gaussian number theory. Our deliverable specializes that arithmetic to the multiplication kernel, supplies machine-checked local contracts and lattice conversion, and includes explicit failure cases that precision certificates can reuse. The existing semantic-child work [4, 5] already obtains a linear semantic guard. Our result sharpens a completed C -child’s denominator enclosure; it does not lower that guard’s asymptotic exponent.

2 A one-bit exactness certificate

Definition 2.1. For $z = r + si \in \mathbb{Z}[i]$, define $\rho(z) = (r + s) \bmod 2 \in \{0, 1\}$. An input pair (u, v) is admissible when $\rho(u) = \rho(v)$.

Lemma 2.2. For $z = r + si$, the following are equivalent:

$$\varpi \mid z, \quad r \equiv s \pmod{2}, \quad \rho(z) = 0.$$

When they hold, the exact quotient is

$$z/\varpi = \frac{r + s}{2} + \frac{s - r}{2}i. \quad (3)$$

Proof. Multiplication by ϖ sends (a, b) to $(a - b, a + b)$. Solving these two integer equations gives (3). Both halves are integral exactly when r and s have equal parity. The argument covers negative integers as well. $\square$

Theorem 2.3 (Local butterfly certificate). For arbitrary $u, v \in \mathbb{Z}[i]$, both entries of $C(u, v)$ belong to $\mathbb{Z}[i]$ if and only if $\rho(u) = \rho(v)$. On this admissible pair lattice, C preserves admissibility and $C^2(u, v) = (v, u)$.

Proof. The two numerators in the ϖ representation are $iu + v$ and $u + iv$. Since $i \equiv 1 \pmod{\varpi}$, both have residue $\rho(u) + \rho(v)$ in $\mathbb{F}_2$. Lemma 2.2 proves the equivalence. Write $u = a + bi$ and $v = c + di$, and $(r, s) = C(u, v)$. For an admissible pair,

$$\Re r + \Im r = a + d, \quad \Re s + \Im s = b + c.$$

Their difference is even precisely under the incoming condition. Finally the numerator matrix in (1) squares to $\varpi^2 X$, where X swaps the two coordinates. Thus $C^2 = X$. The integer implementation satisfies the same identity under its proved exactness predicate. $\square$

The certificate is weaker than comparing real parities and imaginary parities separately. For example,

$$C(1, i) = (1 + i, 0)$$

is integer-exact, although the real components of the two inputs have different parities. Conversely $C(1, 0)$ has half-integer real and imaginary components; treating its integer quotients as exact is invalid.

Proposition 2.4 (A retained shared defect bit). *Let $N_1, \dots, N_4$ be the four integer coordinates of $2C(u, v)$, and put $\varepsilon = (\Re u + \Im u + \Re v + \Im v) \bmod 2$. Then $\varepsilon \in \{0, 1\}$ and, for every j ,*

$$N_j = 2\lfloor N_j/2 \rfloor + \varepsilon.$$

Moreover $\varepsilon = 0$ exactly when the local certificate passes.

Proof. The four numerators are $a - b + c + d$, $a + b - c + d$, $a + b + c - d$, and $-a + b + c + d$. Each has parity $a + b + c + d$. Euclidean division by two gives the identity, including for negative numerators. Theorem 2.3 identifies the zero-defect case. $\square$

Consequently one retained bit reconstructs all four local half-integer components from their integer floors. This is an explicit information-preserving residual, rather than an assumption that XOR can recover discarded integer carries. A later routing operation must transport or replicate this shared information as needed; the proposition establishes no compression or tape-cost saving for an arbitrary schedule.

Proposition 2.5 (Phase and routing compatibility). *The residue satisfies $\rho(iz) = \rho(-z) = \rho(z)$ and $\rho(u + v) = \rho(u) + \rho(v)$ in $\mathbb{F}_2$. Gaussian-unit phases preserve the ideals $\varpi^k \mathbb{Z}[i]$ and binary grids. A permutation of whole coefficient records preserves their common grid and transports their residue labels.*

Proof. Negation and multiplication by i change or exchange signs of integer coordinates, which do not change their parity sum. Addition gives the quotient-ring addition law. Units are invertible, and permutations change neither coefficient values nor the stated common lattice. $\square$

These rules fit the source/sink signs and fourth-root phases surrounding forward and inverse children. They do not authorize reinterpretation of temporary XOR encodings as coefficients: the upstream contract restores those encodings before coefficient arithmetic resumes.

3 The sharp completed-child lattice

Let $C_D = C^{\otimes D}$, with $C_0 = 1$. For addresses $x, y \in \{0, 1\}^D$, let $h(x, y)$ be their Hamming distance. Multiplying the diagonal entry i/ϖ for equal bits and the off-diagonal entry $1/\varpi$ for unequal bits gives

$$(C_D)_{x,y} = \frac{i^{D-h(x,y)}}{\varpi^D}, \quad (C_D^{-1})_{x,y} = \frac{i^{h(x,y)}}{\varpi^D}. \quad (4)$$

Every numerator is a Gaussian unit. This identity concerns a completed tensor; a recursive implementation can have additional intermediate denominator growth.

Theorem 3.1 (Return lattice and sharp binary precision). *For inputs in $2^{-p}\mathbb{Z}[i]$, every completed forward or inverse D -axis tensor output lies in*

$$2^{-p}\varpi^{-D}\mathbb{Z}[i] \subseteq 2^{-(p+\lceil D/2 \rceil)}\mathbb{Z}[i].$$

The additional binary denominator exponent $\lceil D/2 \rceil$ is the least universal one. When D is odd, the numerators in this binary grid have equal real and imaginary parity.

Proof. Equation (4) puts each output in the first lattice. If $D = 2q$, then

$$\varpi^{-D} = 2^{-q}(-i)^q.$$

If $D = 2q + 1$, then

$$\varpi^{-D} = 2^{-(q+1)}(-i)^q(1-i).$$

The odd numerator is divisible by ϖ , hence has equal coordinate parity. For sharpness, apply the transform to an impulse of value 2^{-p} . Each output is a unit times $2^{-p}\varpi^{-D}$. After scaling by $2^{p+\lceil D/2 \rceil}$, its numerator is a unit for even D and a unit times $1-i$ for odd D . In either case its coordinates are not both even. For positive D it cannot lie on the binary grid with one fewer bit. The same impulse argument applies to the inverse. $\square$

The corresponding positive ideal identities are

$$\varpi^{2q}\mathbb{Z}[i] = 2^q\mathbb{Z}[i], \quad \varpi^{2q+1}\mathbb{Z}[i] = \{2^qz : z \in \mathbb{Z}[i], \rho(z) = 0\}. \quad (5)$$

The Lean development proves these equalities constructively and proves the inverse-lattice conversion through integer cross-multiplication. It does not need a quotient field or any analytic assumption.

Proposition 3.2 (Final normalization has a different bound). *For $H_D = H_0^{\otimes D}$, the least universal additional binary denominator exponent is D . The C_D return bound alone does not justify using $\lceil D/2 \rceil$ bits for H_D .*

Proof. The final adapter contributes $b^D = \varpi^{-D}$, in addition to the ϖ^{-D} from C_D . Their product is $\varpi^{-2D} = (-i)^D 2^{-D}$. Equivalently, every H_D entry is $(-1)^{x \cdot y} / 2^D$, and an impulse gives the sharp witness. For odd D , separately bounding both factors by $\lceil D/2 \rceil$ gives $D+1$ bits; retaining the correlated Gaussian parity recovers the exact D -bit bound. $\square$

Even at $D = 1$, C admissibility does not suffice for H_0 integrality. The example $(1, i)$ is admissible for C , whereas $H_0(1, i) = ((1+i)/2, (1-i)/2)$ is not Gaussian-integral. Exact H_0 division requires equal parity of the two real components and equal parity of the two imaginary components. This stronger boundary predicate is also checked in Lean.

4 A total certificate interface

Represent a Gaussian dyadic value by (z, e) with decoded value z/ϖ^e . The exponent is a denominator obligation, not information that can be discarded. For inputs at a common exponent e , set $n_L = iu + v$ and $n_R = u + iv$. Use the following total rule:

Predicate	Stored output numerators	Output exponent
$\rho(u) = \rho(v)$	$n_L/\varpi, n_R/\varpi$	e
$\rho(u) \neq \rho(v)$	n_L, n_R	$e + 1$

The success branch uses the exact quotient of Lemma 2.2. The fallback retains both integer numerators. Either branch represents the same exact C output. The Lean implementation expresses equality of representations by cross-multiplying their ϖ powers, and proves the rule's refinement of the unconditional numerator representation.

An implementation with a single fixed-width stream need not allocate a variable-size record per coefficient. It can use the parity predicate as a certificate or audit assertion while retaining the existing uniform guarded buffer. If it instead changes a common exponent or rewrites a stream, those scans and copies must be included in its tape cost. Per-pair local exponents do not by themselves supply the common grid required by a later addition or recursive call.

For unequal incoming tags e, f , first use $E = \max\{e, f\}$ and multiply the corresponding numerators by ϖ^{E-e} and ϖ^{E-f} . Lean proves that this alignment preserves their represented values and that the aligned total step satisfies the same refinement contract. The formalization assigns no running-time cost to the alignment.

A rejected global shortcut. An initial promise that all coefficients have the same residue does not certify all later pairings. Four inputs $(1, 1, 1, i)$ have residue one. Pairing the first two and last two gives outputs $(1, 1, 1 + i, 0)$. The next layer pairing their first outputs encounters residues one and zero, so its division fails. The Lean file proves this changed-pairing obstruction. It is a counterexample to a proposed unchecked shortcut, not a counterexample to any cited multiplication theorem.

5 Formalization and validation

The accompanying Lean files use Lean 4.31.0 and the bundled `Std` library, with no Mathlib download. Their Gaussian integers are signed integer pairs. All proof obligations are discharged by the kernel; the source contains no omitted proofs or project-specific axioms. Integer division is treated as a computational operation whose exactness must be proved, not presumed.

Claim	Formal artifact
Exact ϖ division and residue characterization	<code>DyadicCertificates.lean: dividePi_exact_iff, residue_zero_iff</code>
Exact C input predicate and stable pair lattice	<code>butterfly_integral_iff, butterfly_exact, butterfly_sameResidue</code>
C^2 swap identity and phase residue laws	<code>butterfly_square, residue_mulI, residue_add</code>
Four coordinates reconstructed by one defect bit	<code>sharedDefect_is_bit, sharedDefect_zero_iff, shared_defect_reconstruction</code>
Total fallback and unequal-tag alignment	<code>certifiedButterfly_refines, raiseTo_sameValue, alignedButterfly_refines</code>
Even/odd Gaussian ideal hierarchy	<code>GaussianLattice.lean: repeatPi_even, pi_even_ideal, pi_odd_ideal</code>
Inverse-lattice binary conversion and witnesses	<code>binary_grid_conversion, odd_binary_numerator_parity, even_grid_sharp, odd_grid_sharp</code>
Odd-level normalization cancellation	<code>two_pi_factors_binary</code>
Incorrect global parity shortcut	<code>changed_pairing_obstruction</code>

The tensor coefficient formula (4) is established by the written tensor-product argument above and independently compared to a staged exact evaluator. The Lean lattice theorem is quantified over arbitrary Gaussian numerators and denominator exponents; connecting it to a concrete recursive network additionally requires that network’s completed-child semantic contract. The full recursive schedule, tape movement, magnitude bounds, analytic inequalities, and multiplication exponent are outside this formalization.

The standard-library Python audit uses exact rational arithmetic to compare closed-form and staged tensors, local division certificates, phases, inverse operations, and the normalized adapter. It checks 3,721 quotient cases, 13,122 forward/inverse pair cases, 72 C -tensor cases with 4,088 output values, and 36 normalized cases with 2,044 output values, at dimensions zero through eight. Both transforms have nine impulse sharpness probes; four negative controls reject deliberately false predicates. Its recorded output is included in `verification.json`. These finite checks supplement the universal Lean proofs; they do not replace them.

6 How the initiative can use the result

The most direct insertion site is the completed-child denominator enclosure in the semantic precision note of PR #23 [4]. Its convenient $2^{-D}\mathbb{Z}[i]$ enclosure for a C_D child can be refined to $\varpi^{-D}\mathbb{Z}[i]$, or to the sharp binary enclosure in Theorem 3.1. Keep its intermediate magnitude and guard arguments, arbitrary-scratch contract, source-frame identities, and no-rounding requirement. The sharper return lattice supplies an additional testable obligation at the child boundary.

A useful integration has three steps. First attach the one-bit residue to each integer numerator and use Theorem 2.3 before exact quotient operations. Second verify the completed child against its expected ϖ denominator and phase; whole-record routing transports the annotations. Third verify the final $b^D S_D$ adapter with Proposition 3.2. The negative controls reject a forgotten adapter, a componentwise predicate used as a necessary condition for C , and an initial parity promise reused across changed pairings.

The independently supplied Lean arithmetic certificates [6, 3] check valuable obligations elsewhere in the initiative. This note supplies a complementary Gaussian arithmetic component. It establishes neither that every live branch lacks such a component nor that the full proposed multiplication construction satisfies all its prerequisites. The algebra and local certificate do not depend on the truth of the upstream asymptotic theorem.

Scope of the contribution. The transferable insight is that a binary guard can carry one additional Gaussian parity constraint. That constraint is useful both for accepting exact local divisions and for exposing a missing denominator at an interface. We provide the proof artifact and a proposed integration point. We report no new κ , no wall-clock speedup, and no replacement for the communal network or analytic proofs.

References

- [1] OpenAI. *Integer multiplication below $n \log n$* . 23 September 2026. Pinned source commit `adc7f1241b42e322a6451854ab7e4b4c146bf78a`. [Butterfly-layer source](#).
- [2] Douglas Colkitt. *integer-mult-bounds*: conditional multiplication bounds and communal extensions. Pinned commit `1a74950ce5074b848243ba89d8022fbadba66105`. [Repository](#).
- [3] Swapnil Jain. *Integer multiplication: a conditional witness above 2^{-16}* . Pinned commit `c2c2f279d93643e5ff3fe121a0fbc68e0e6f4007`, 8 October 2026. [Scope and reproduction instructions](#).
- [4] Zhihao Chen (jacklightChen). *Semantic precision and bulk resampling*, communal PR #23. Pinned commit `661ebabc1076c9f525d7ce4967c876b203fa9827`. [Semantic precision note](#).
- [5] RaD (hipotures). *Semantic-child guard review*, attributed source vendored in PR #23. [Review and exact controls](#).
- [6] princezuda. *Lean certificate check*, communal PR #26. Pinned commit `f764fb98208506354730f6f108a8c87fb679432e`. [Declared formal scope](#).
- [7] Matthew Amy, Andrew N. Glaudell, and Neil J. Ross. *Number-Theoretic Characterizations of Some Restricted Clifford+T Circuits*. Quantum 4, 252 (2020). [doi:10.22331/q-2020-04-06-252](#); [arXiv:1908.06076](#).